

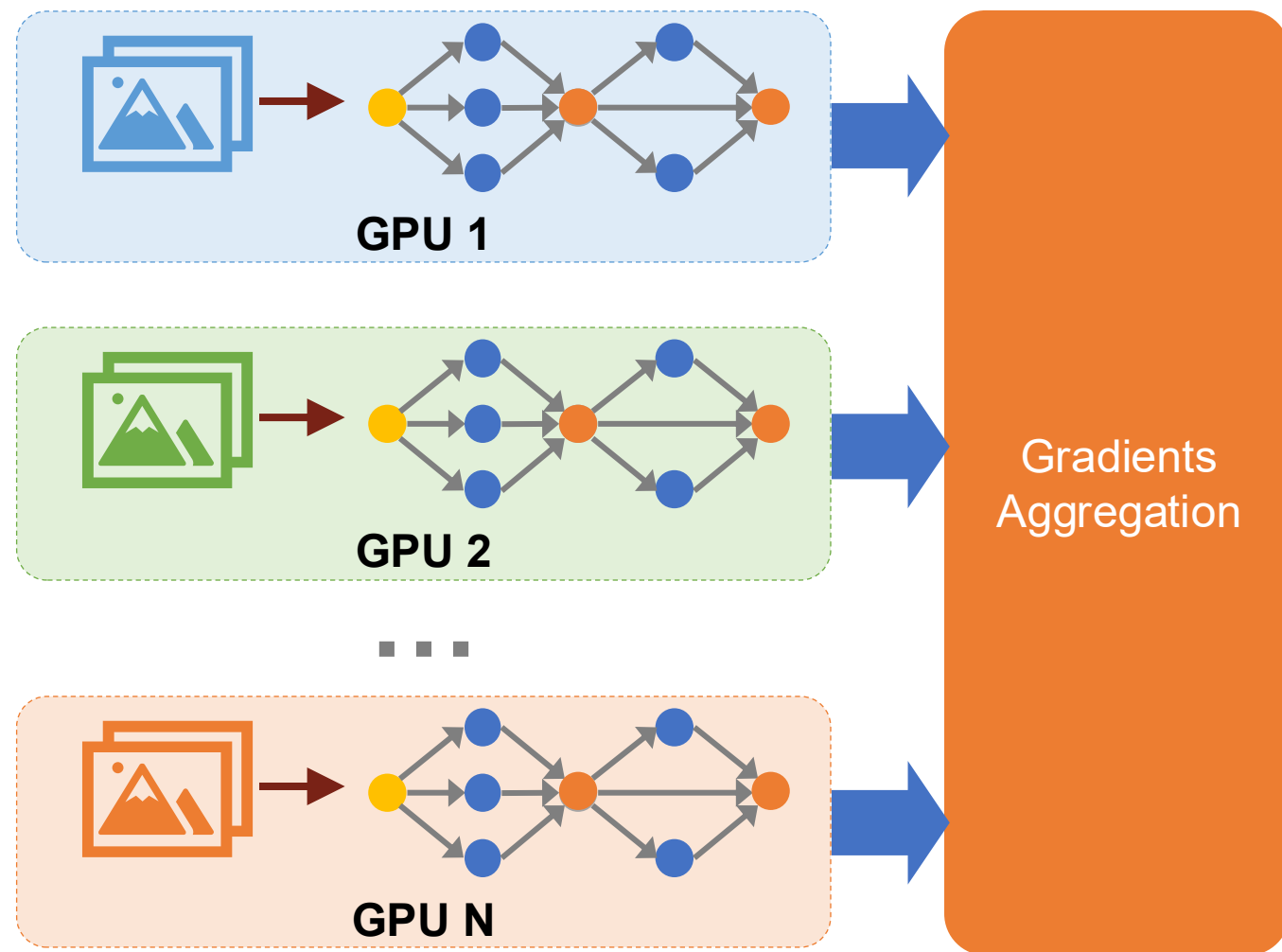
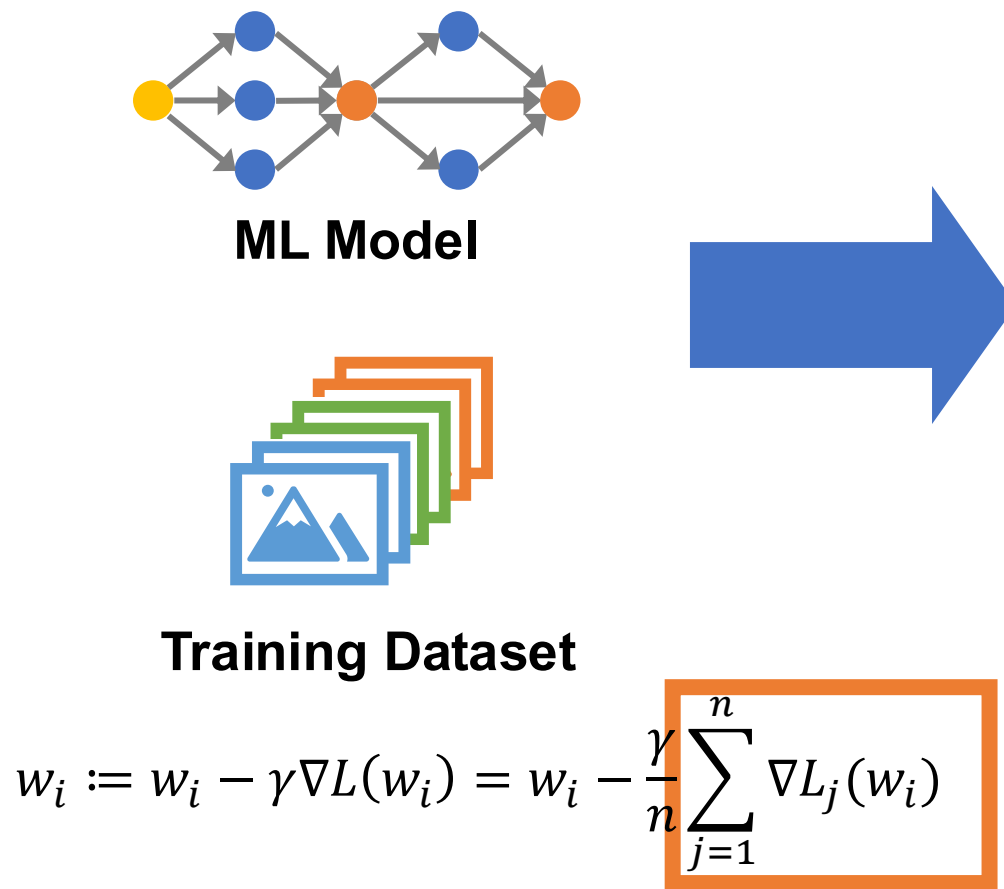
15-442/15-642: Machine Learning Systems

Parallelization Part 2 (Model and Pipeline Parallelism)

Tianqi Chen and Zhihao Jia

Carnegie Mellon University

Recap: Data Parallelism



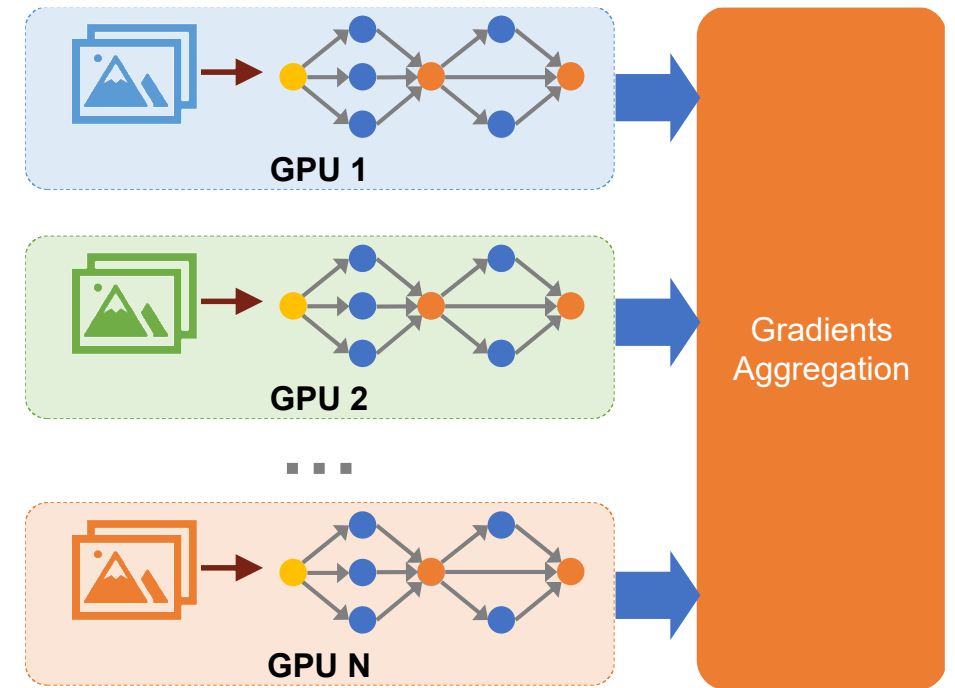
1. Partition training data into batches

2. Compute the gradients of each batch on a GPU

3. Aggregate gradients across GPUs

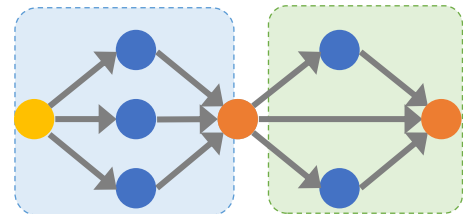
Recap: An Issue with Data Parallelism

- Each GPU saves a replica of the entire model
- Cannot train large models that exceed GPU device memory



Model Parallelism

- Split a model into multiple subgraphs and assign them to different devices

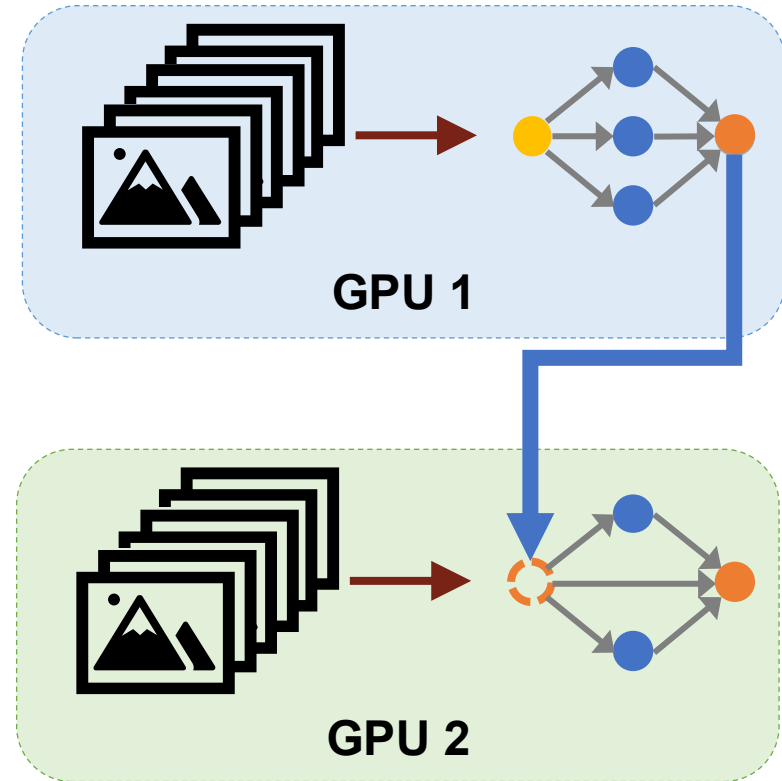


ML Model



Training Dataset

**Model
Parallelism**



Transfer
intermediate
results
between
devices

$$w_i := w_i - \gamma \nabla L(w_i) = w_i - \frac{\gamma}{n} \sum_{j=1}^n \nabla L_j(w_i)$$

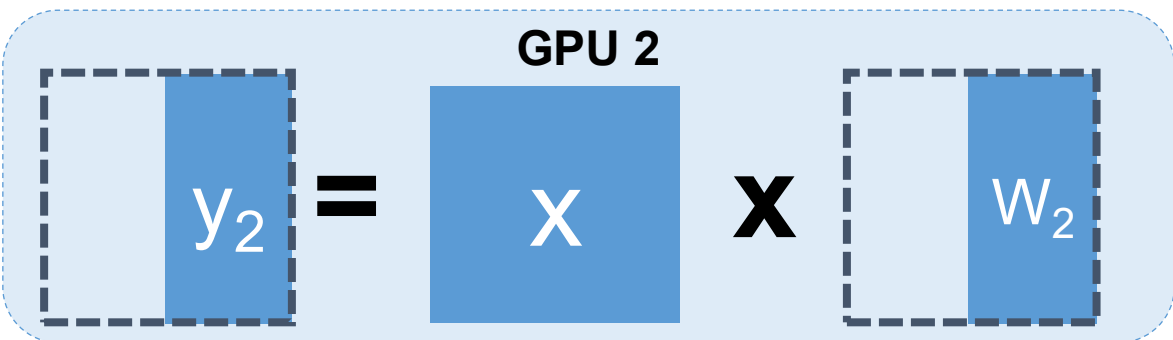
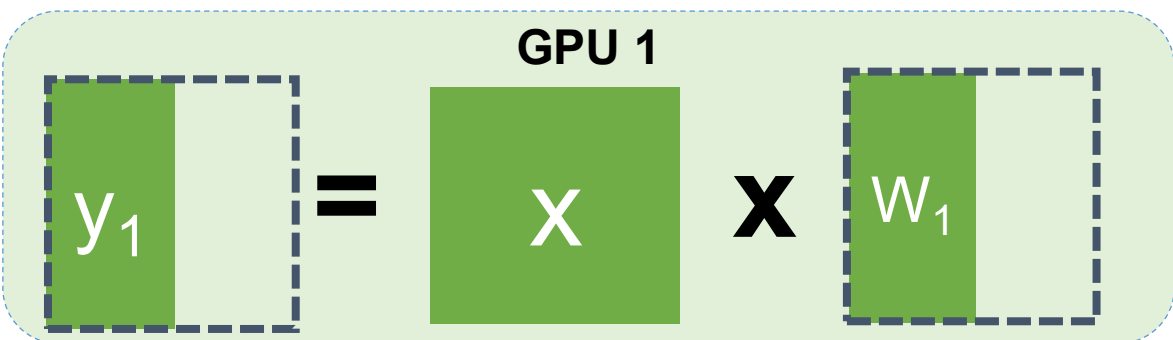
How to parallelize DNN Training?

- Data parallelism
- Model parallelism
 - Tensor model parallelism
 - Pipeline model parallelism

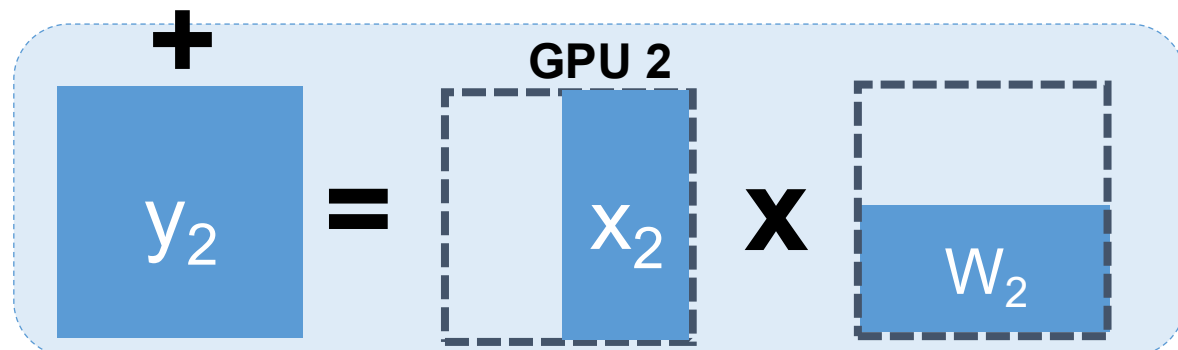
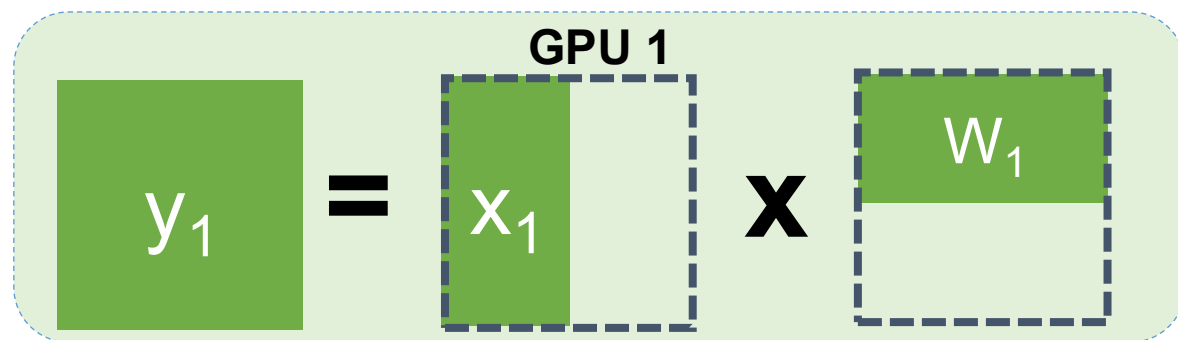
Tensor Model Parallelism

$$\begin{array}{c} \text{output} \end{array} = \begin{array}{c} \text{input} \end{array} \times \begin{array}{c} \text{parameters} \end{array}$$

- Partition parameters/gradients *within* a layer



Tensor Model Parallelism (partition output)

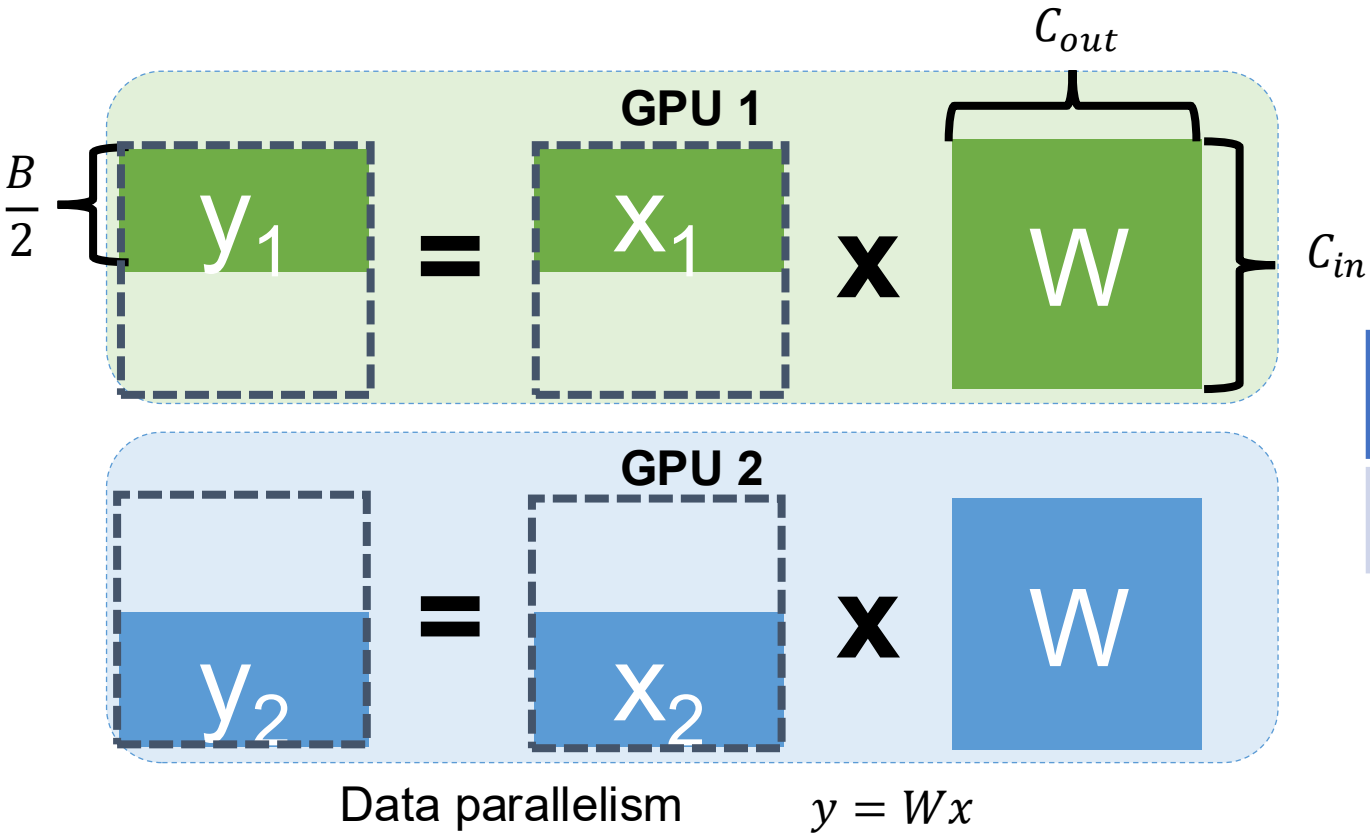


Tensor Model Parallelism (reduce output)

$$y = y_1 + y_2$$

Comparing Data and Tensor Model Parallelism

Diagram illustrating full tensor model parallelism on a single GPU. The output tensor y has dimensions B (height) and C_{out} (width). It is equal to the input tensor x (dimensions B and C_{in}) multiplied by the weight tensor W (dimensions C_{in} and C_{out}).

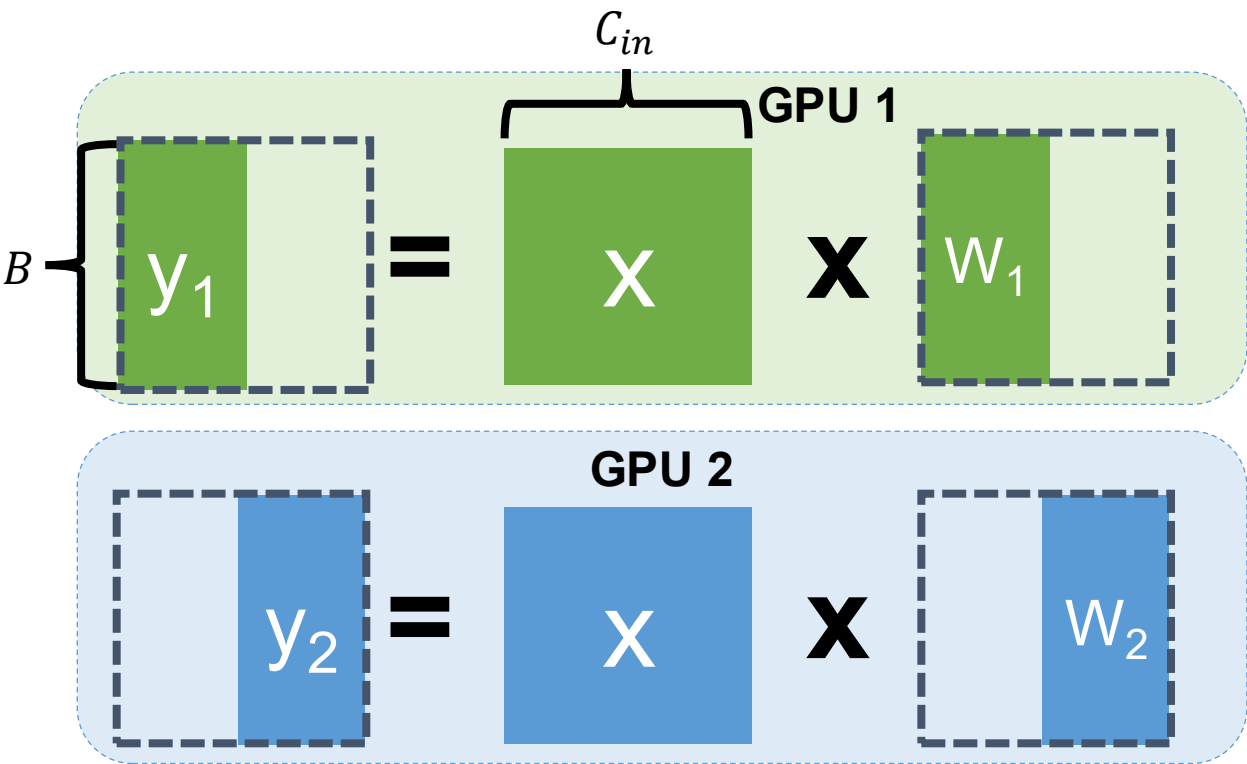


Forward Processing	Backward Propagation	Gradients Sync
0	0	$O(C_{out} * C_{in})$

Communication Cost of Data Parallelism

Comparing Data and Tensor Model Parallelism

$$\begin{matrix} C_{out} \\ \text{ } \\ B \end{matrix} \left[\begin{matrix} y \end{matrix} \right] = \begin{matrix} C_{in} \\ \text{ } \\ B \end{matrix} \left[\begin{matrix} x \end{matrix} \right] \times \begin{matrix} C_{out} \\ \text{ } \\ C_{in} \end{matrix} \left[\begin{matrix} W \end{matrix} \right]$$



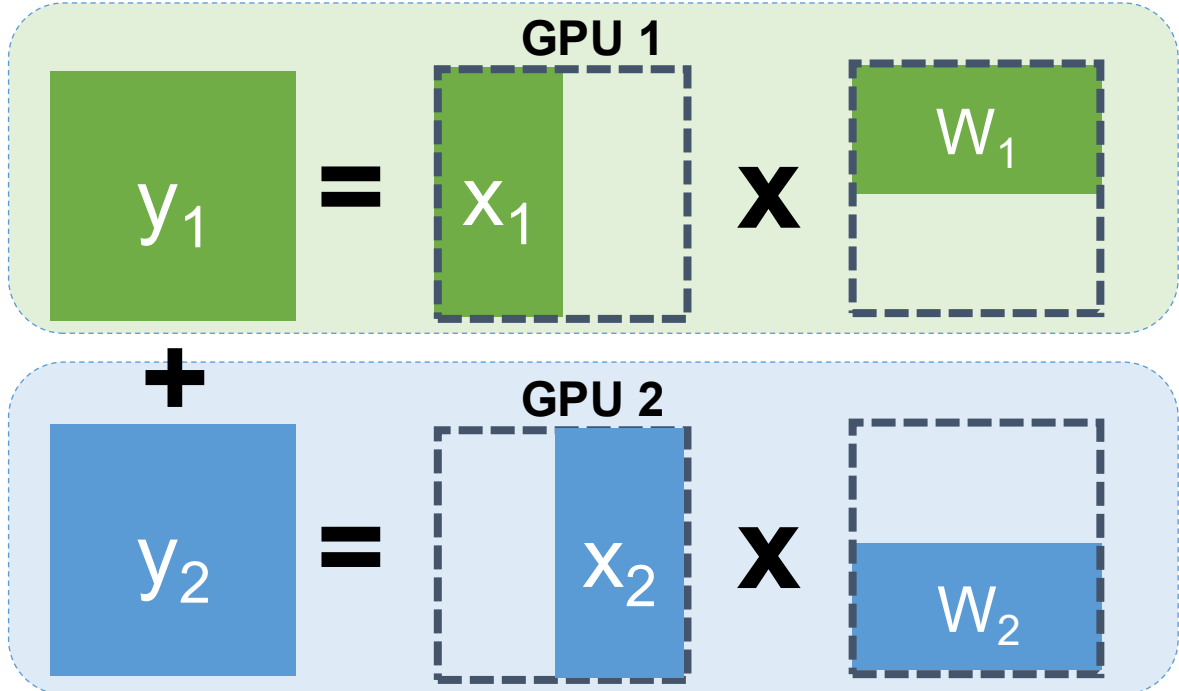
Tensor Model Parallelism (partition output)

Forward Processing	Backward Propagation	Gradients Sync
$O(B * C_{in})$	$O(B * C_{in})$	0

Communication Cost of Tensor Model Parallelism

Comparing Data and Tensor Model Parallelism

Diagram illustrating the dimensions of tensors in a matrix multiplication operation. The output tensor y has dimensions B (height) and C_{out} (width). The input tensor x has dimensions B (height) and C_{in} (width). The weight tensor W has dimensions C_{in} (height) and C_{in} (width). The operation is represented as $y = x \times W$.



Tensor Model Parallelism (Reduce output)

$y = y_1 + y_2$

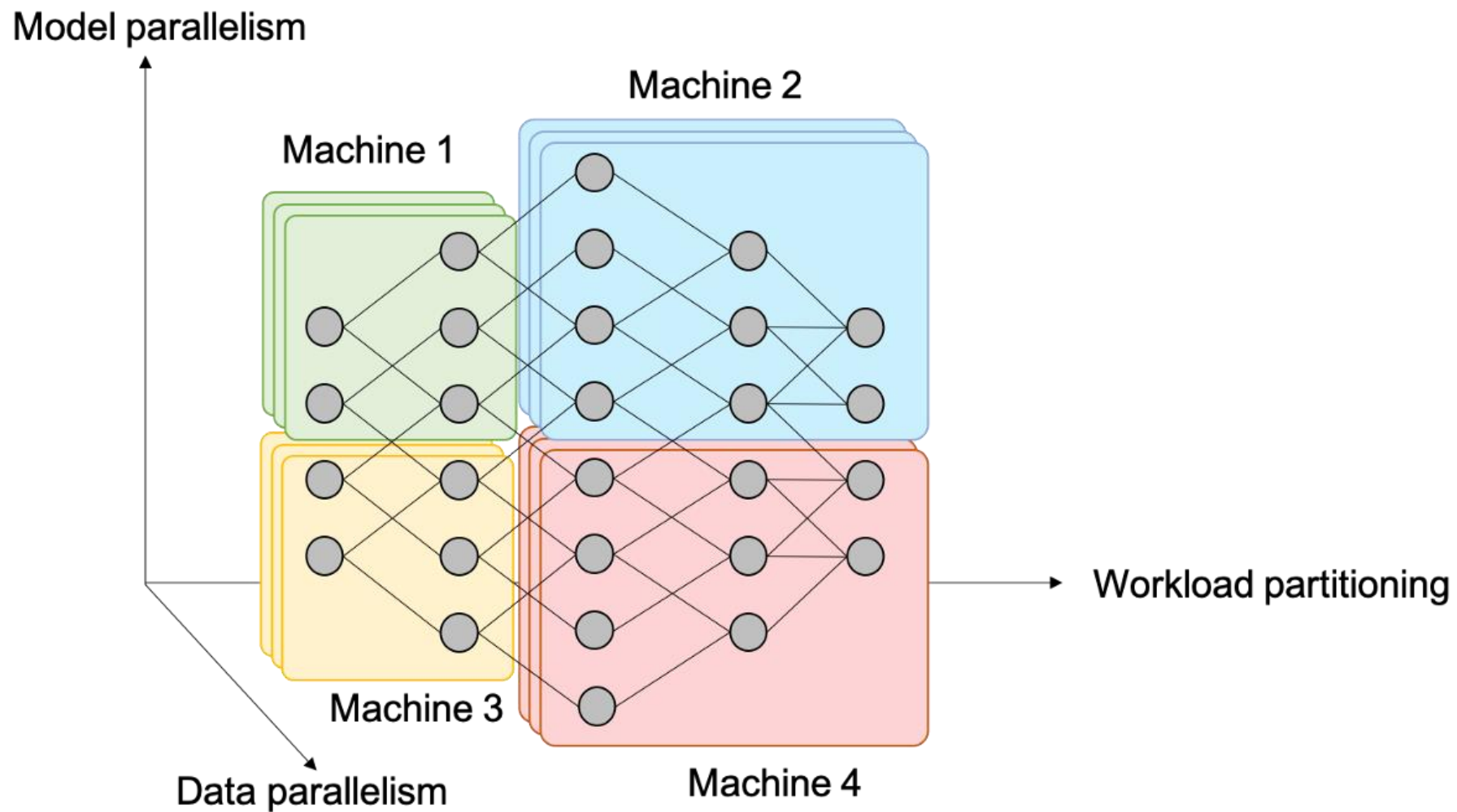
Forward Processing	Backward Propagation	Gradients Sync
$O(B * C_{out})$	$O(B * C_{out})$	0

Communication Cost of Tensor Model Parallelism

Comparing Data and Tensor Model Parallelism

- Data parallelism: $O(C_{out} * C_{in})$
- Tensor model parallelism (partition output): $O(B * C_{in})$
- Tensor model parallelism (reduce output): $O(B * C_{out})$
- **The best strategy depends on the model and underlying machine**

Combine Data and Model Parallelism



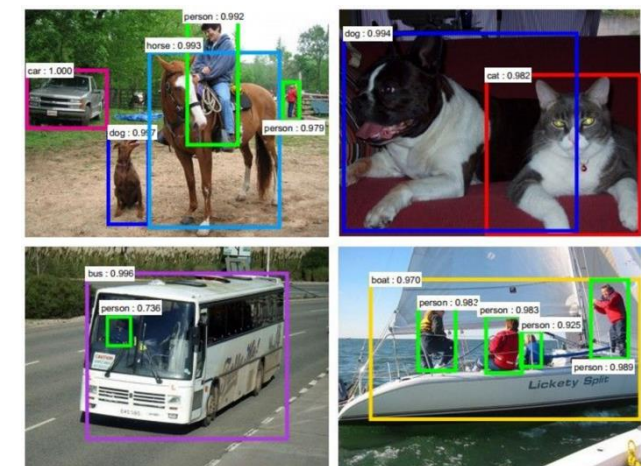
Example: Convolutional Neural Networks



Classification



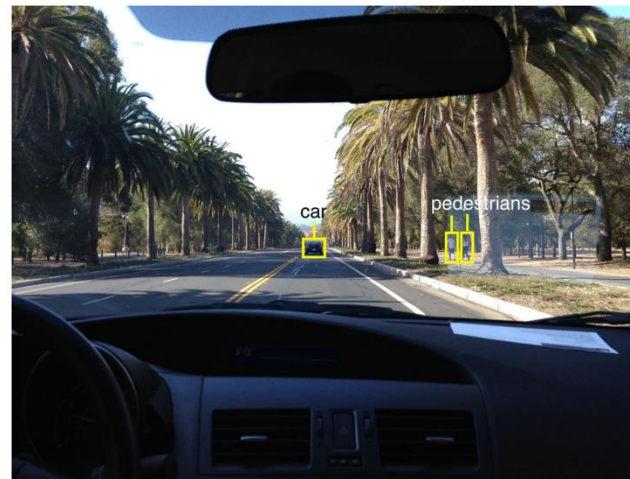
Retrieval



Detection



Segmentation



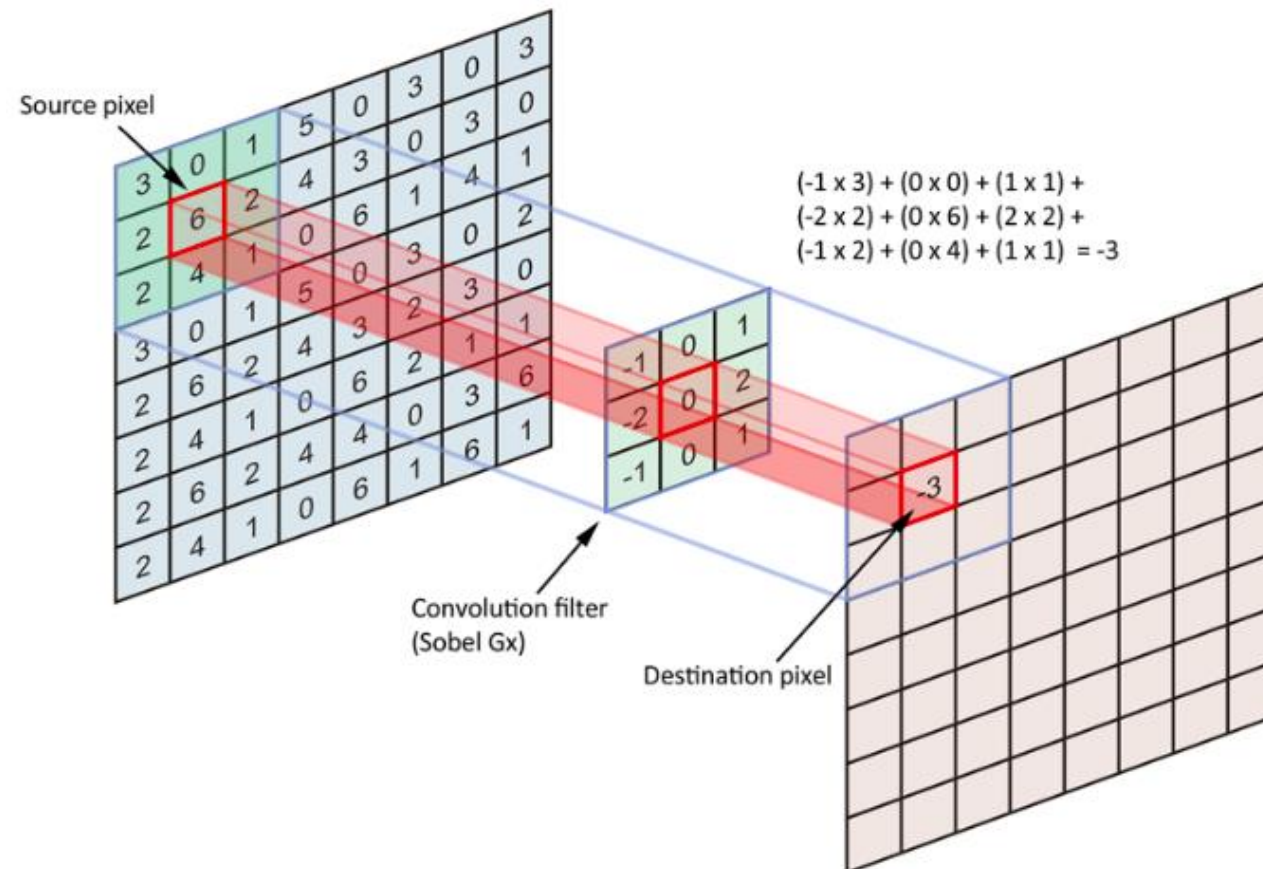
Self-Driving



Synthesis

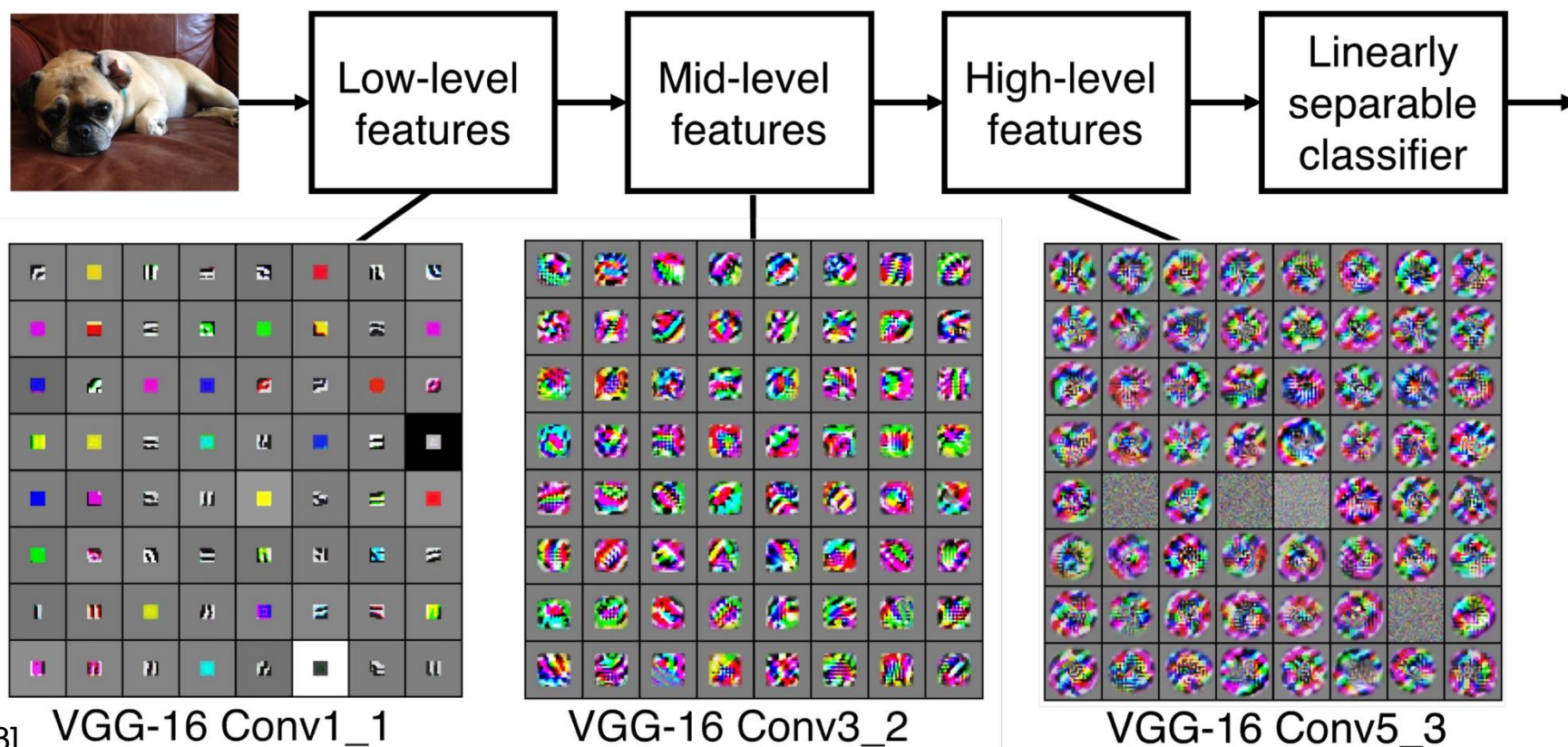
Convolution

- Convolve the filter with the image: slide over the image spatially and compute dot products



CNNs

- A sequence of convolutional layers, interspersed by pooling, normalization, and activation functions



Parallelizing Convolutional Neural Networks

- Convolutional layers
 - 90-95% of the computation
 - 5% of the parameters
 - Very large intermediate activations

Data parallelism

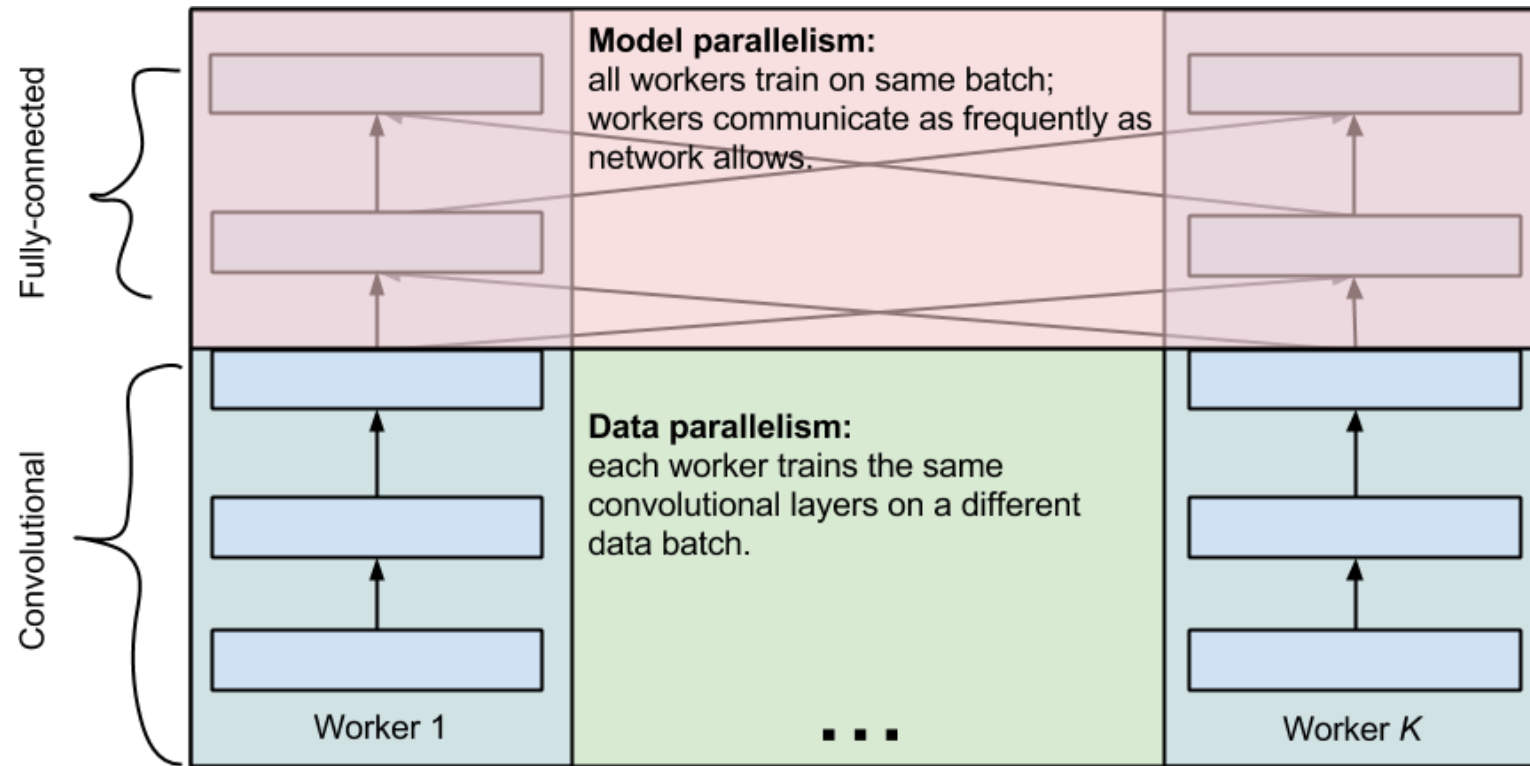
- Fully-connected layers
 - 5-10% of the computation
 - 95% of the parameters
 - Small intermediate activations

Tensor model parallelism

- **Discussion: how to parallelize CNNs?**

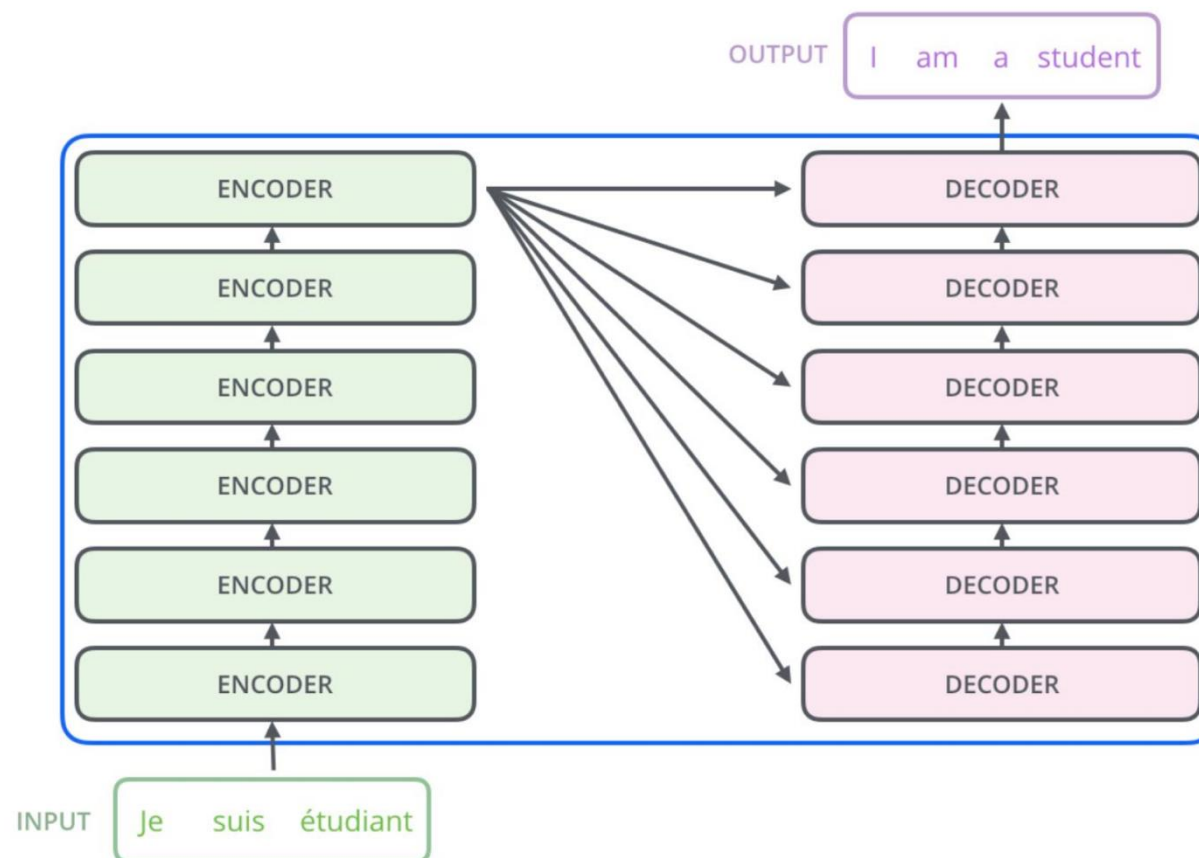
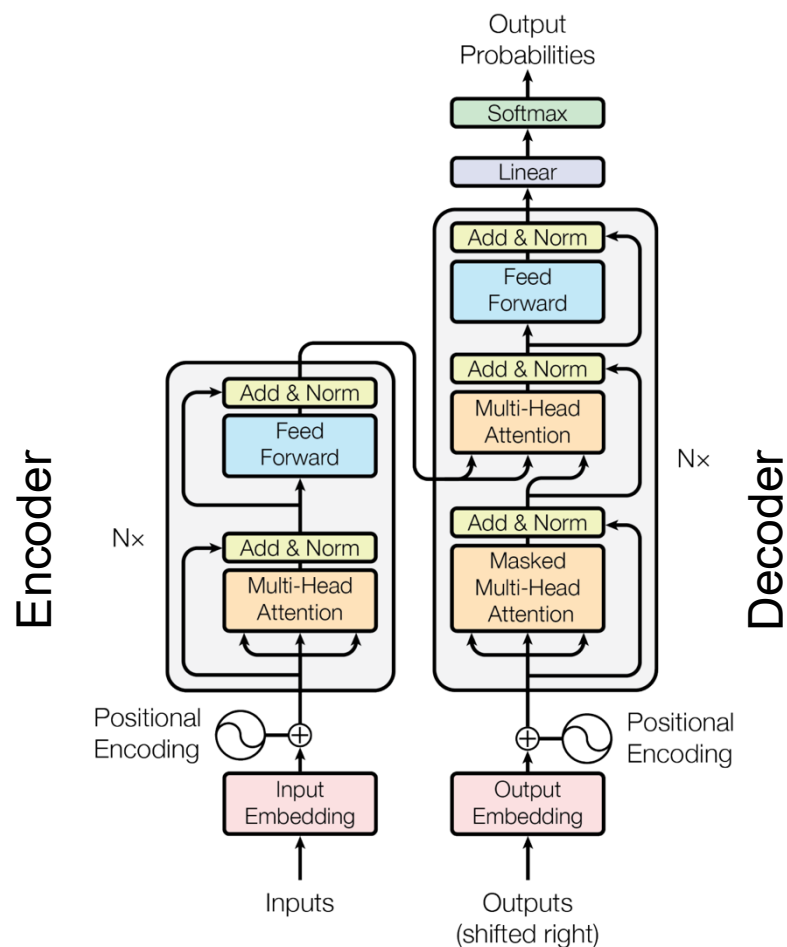
Parallelizing Convolutional Neural Networks

- Data parallelism for convolutional layers
- Tensor model parallelism for fully-connected layers



Example: Parallelizing Transformers

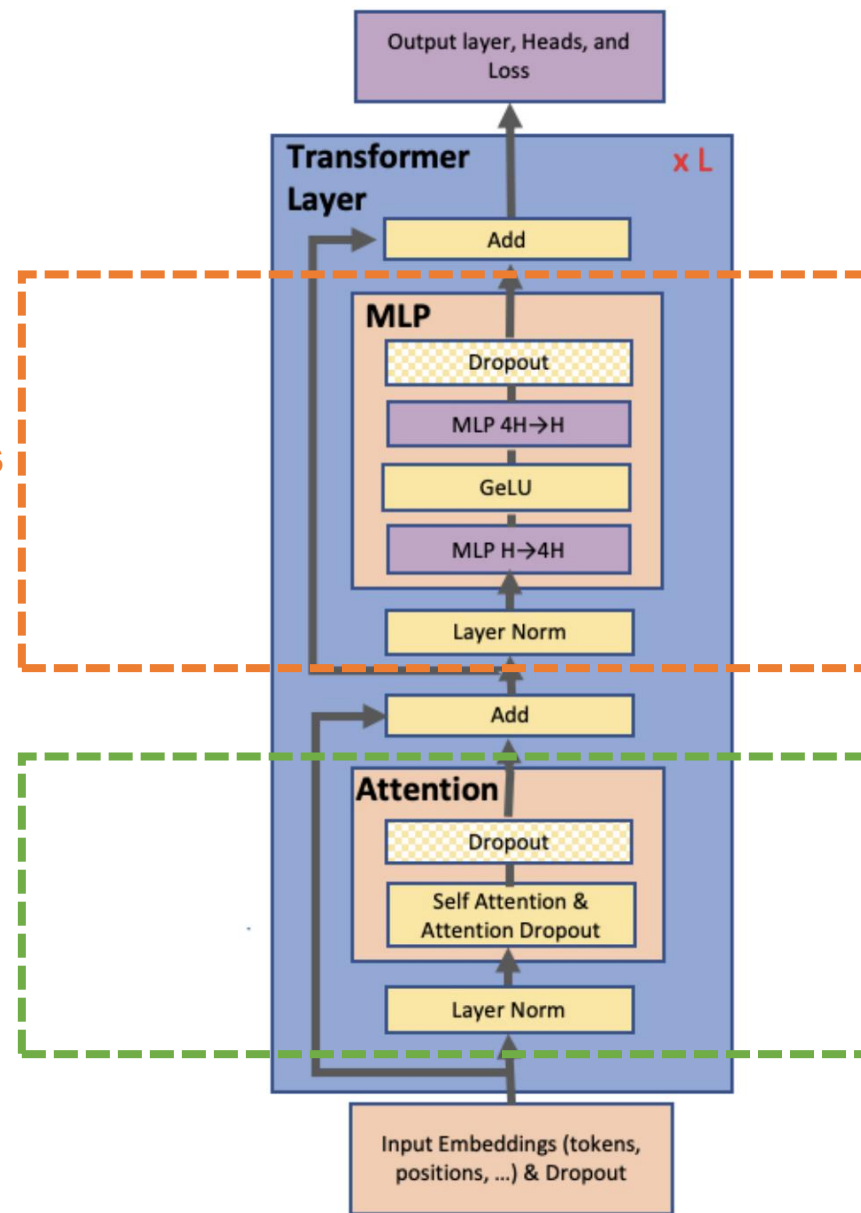
- Transformer: attention mechanism for language understanding



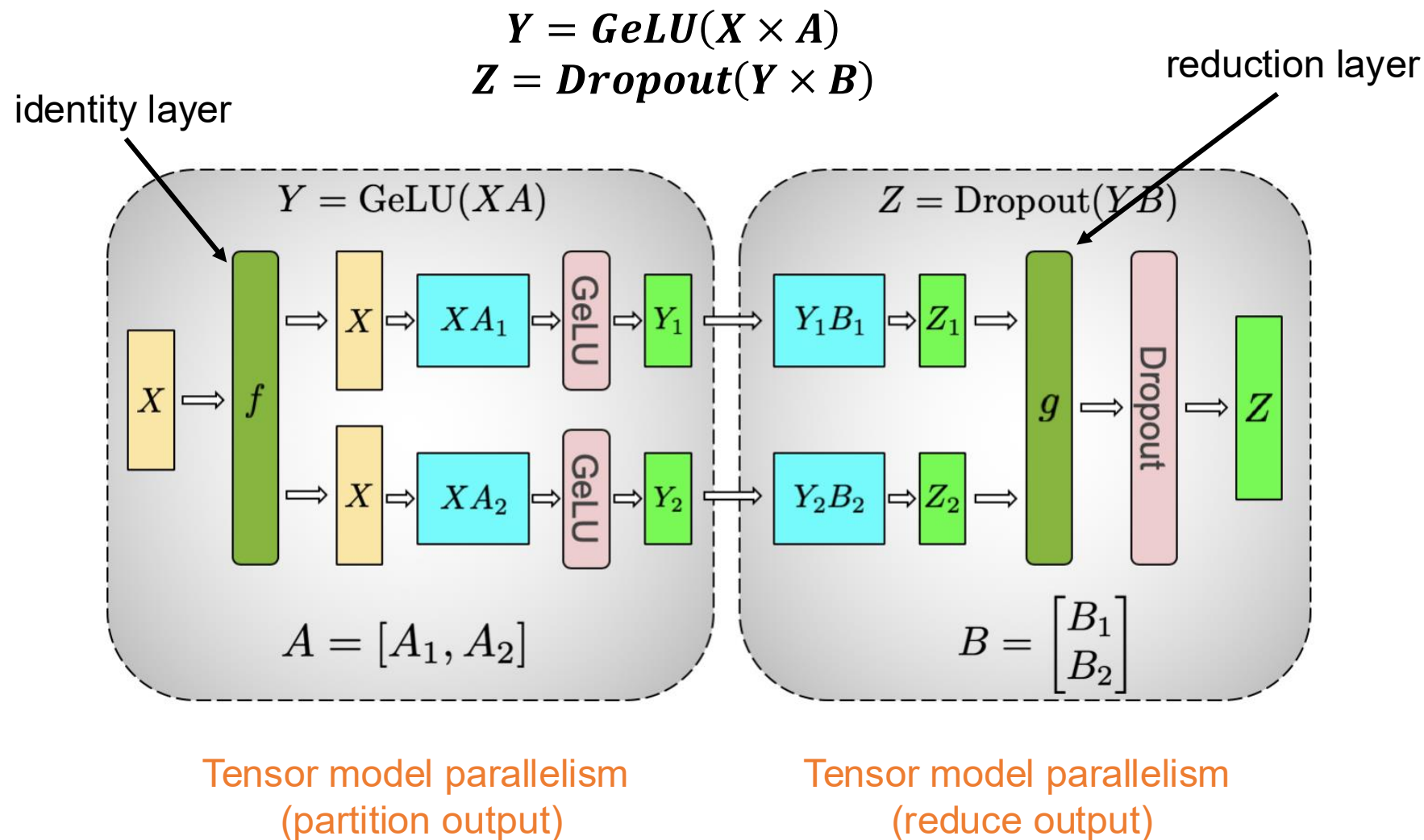
A Single Transformer Layer

Fully-Connected Layers

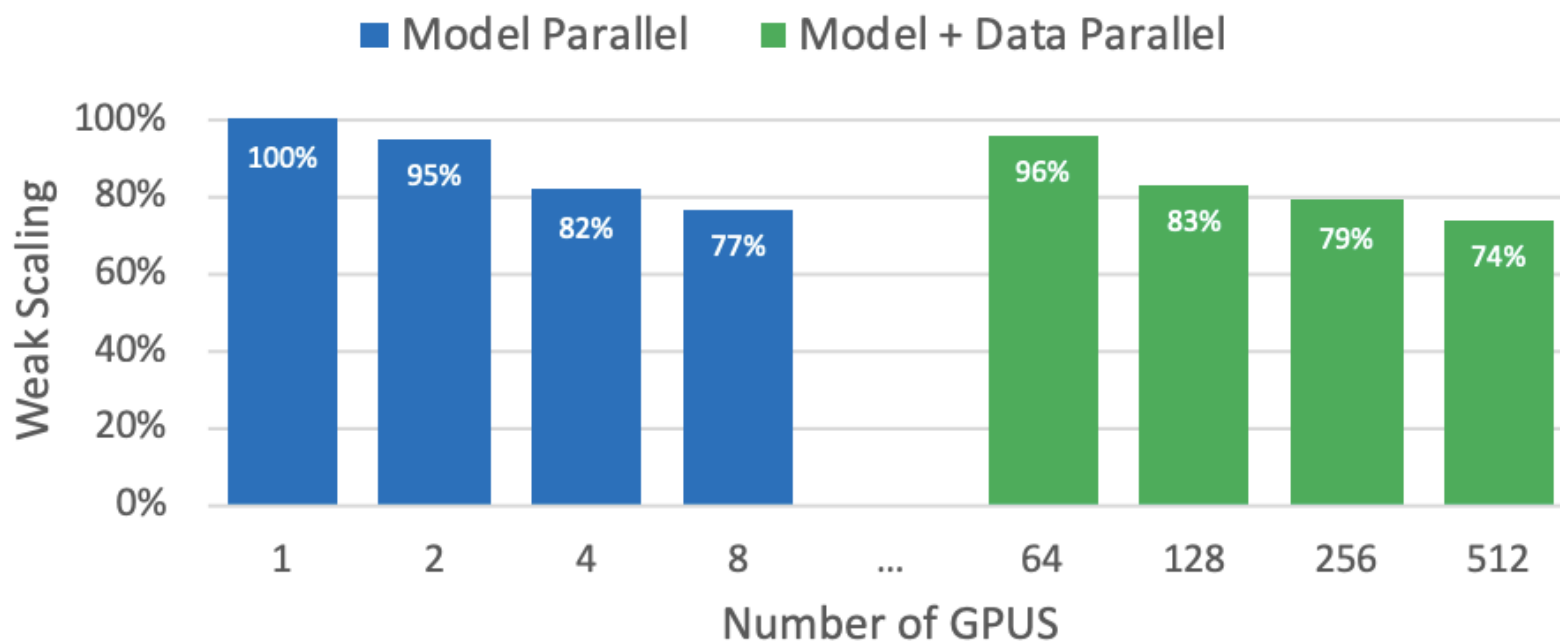
Self-Attention Layers



Parallelizing Fully-Connected Layers in Transformers



Parallelizing Transformers



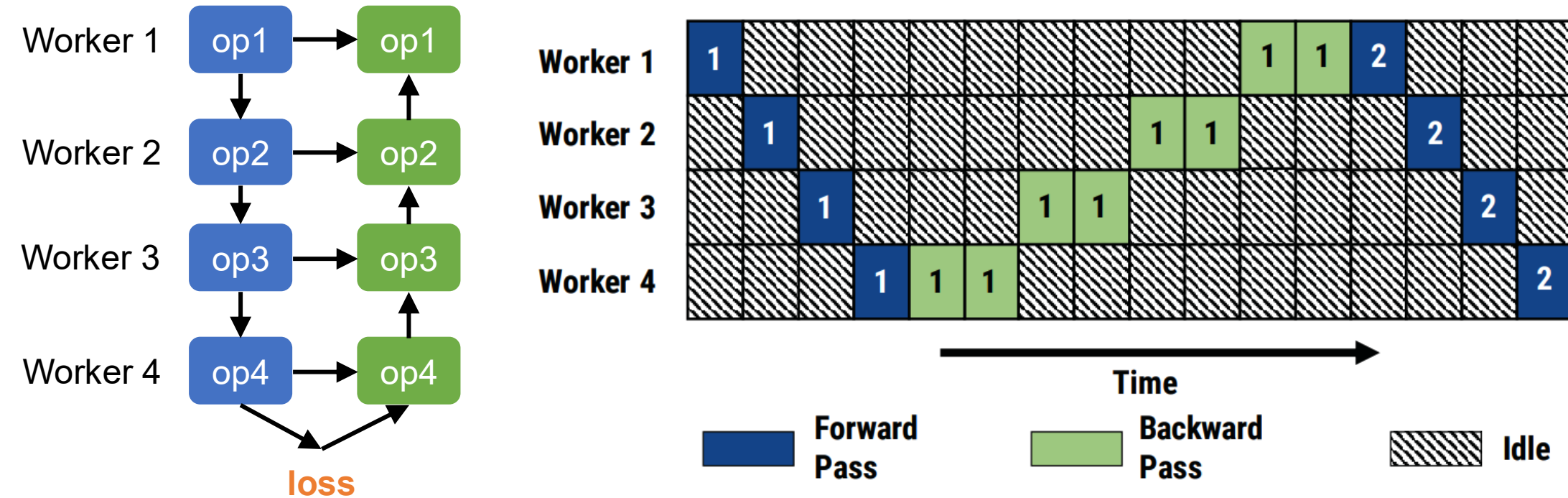
Scale to 512 GPUs by combining data and model parallelism

How to parallelize DNN Training?

- Data parallelism
- Model parallelism
 - Tensor model parallelism
 - **Pipeline model parallelism**

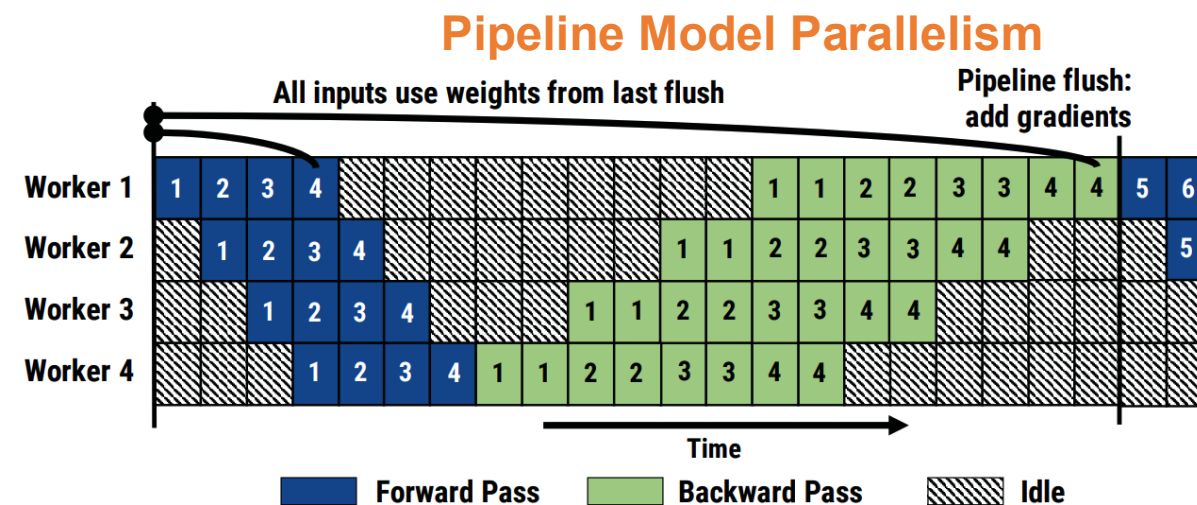
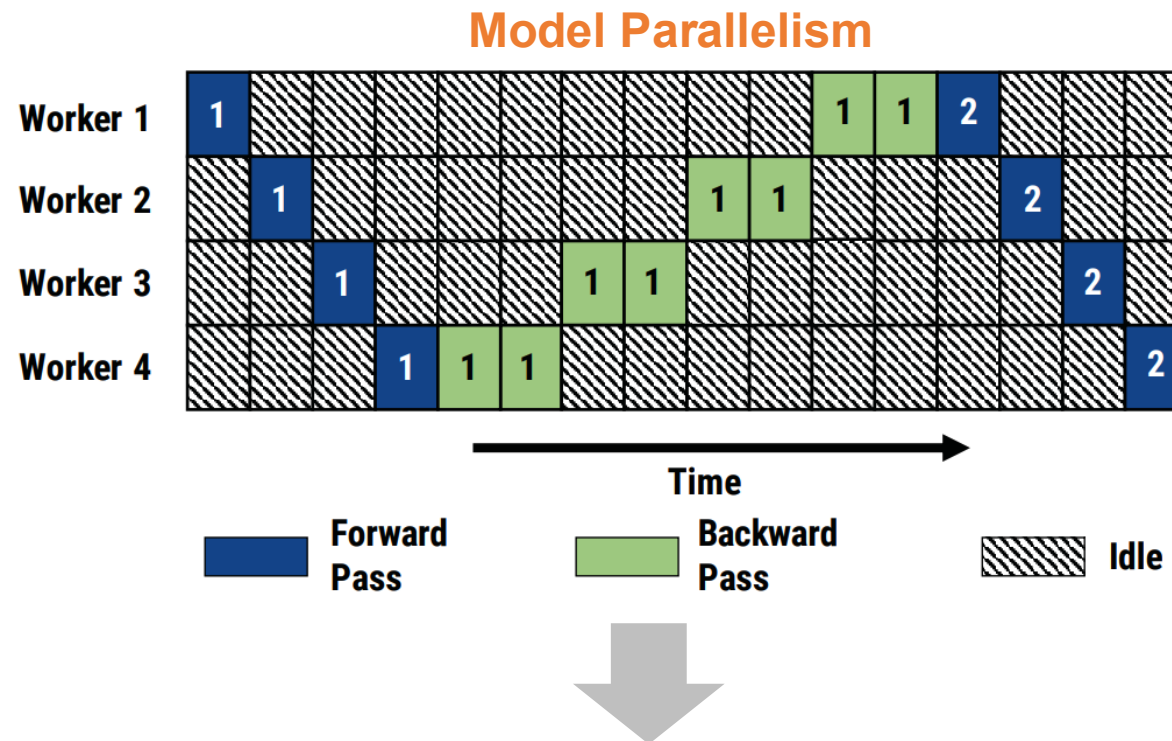
An Issue with Model Parallelism

- Under-utilization of compute resources
- Low overall throughput due to resource utilization



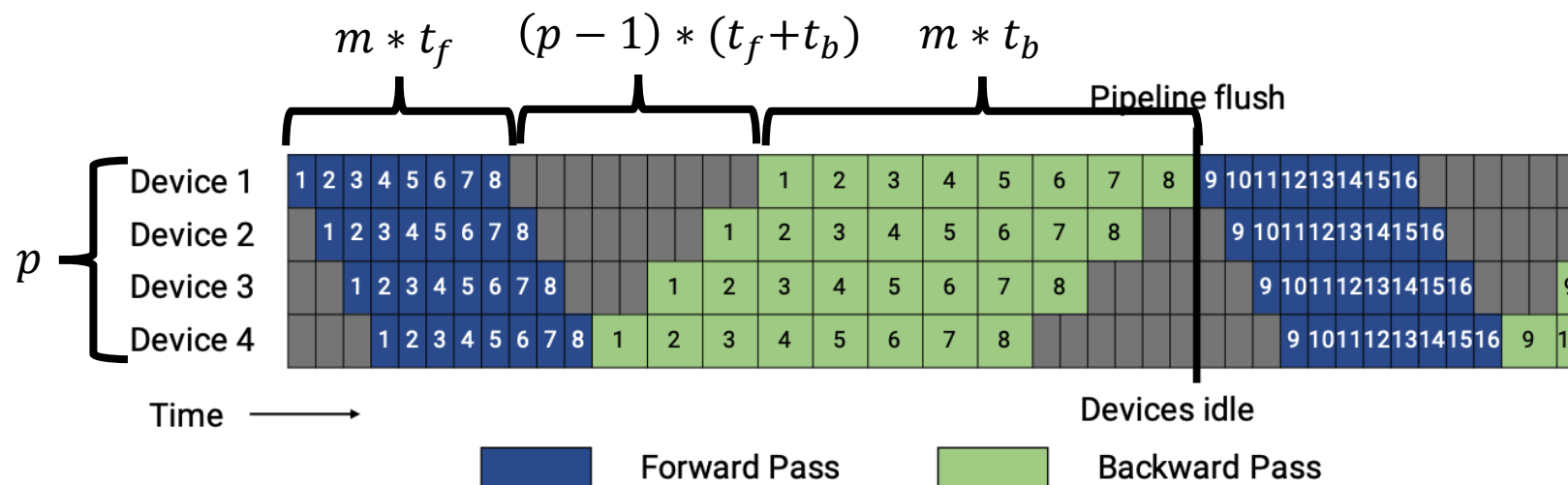
Pipeline Model Parallelism

- **Mini-batch**: the number of samples processed in each iteration
- Divide a mini-batch into multiple **micro-batches**
- Pipeline the forward and backward computations across micro-batches



Pipeline Model Parallelism: Device Utilization

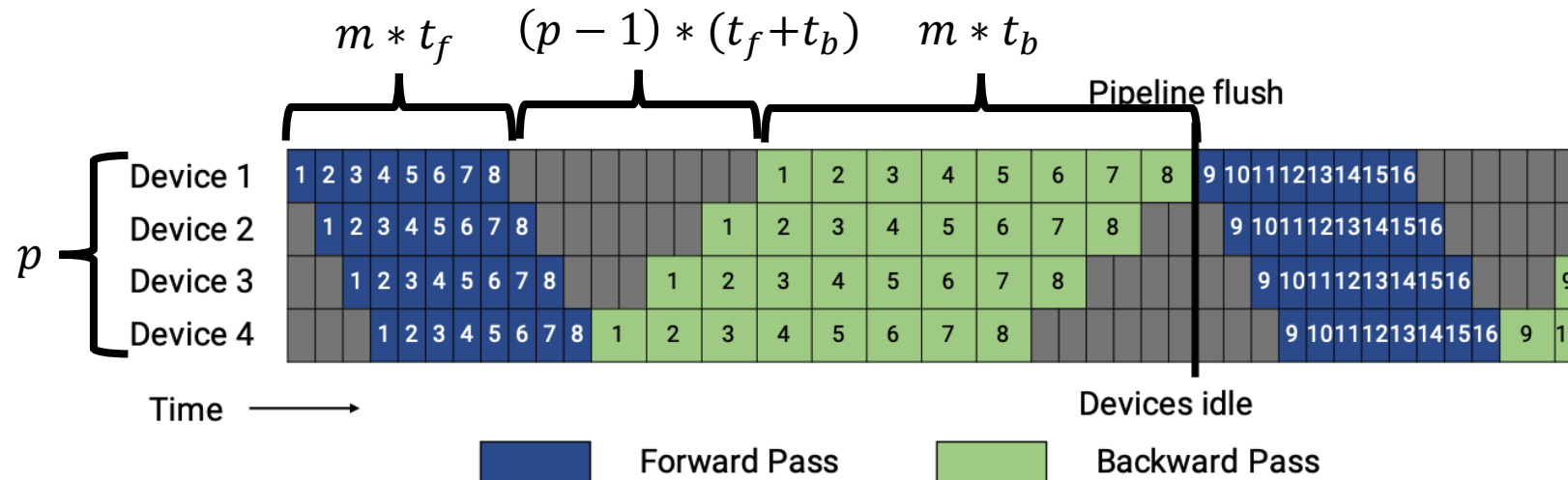
- m : micro-batches in a mini-batch
- p : number of pipeline stages
- All stages take t_f / t_b to process a forward (backward) micro-batch



$$\text{BubbleFraction} = \frac{(p - 1) * (t_f + t_b)}{m * t_f + m * t_b} = \frac{p - 1}{m}$$

Improving Pipeline Parallelism Efficiency

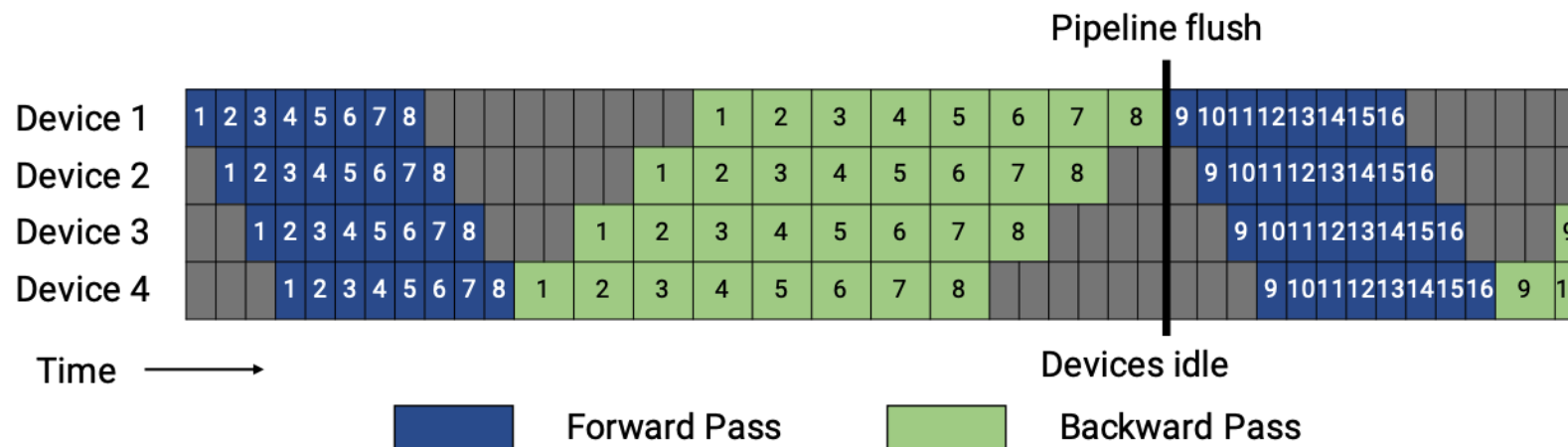
- m : number of micro-batches in a mini-batch
 - Increase mini-batch size or reduce micro-batch size
 - Caveat: large mini-batch sizes can lead to accuracy loss; small micro-batch sizes reduce GPU utilization
- p : number of pipeline stages
 - Decrease pipeline depth
 - Caveat: increase stage size



$$\text{BubbleFraction} = \frac{(p-1) * (t_f + t_b)}{m * t_f + m * t_b} = \frac{p-1}{m}$$

Pipeline Model Parallelism: Memory Requirement

- An issue: we need to keep the intermediate activations of **all micro-batches** before back propagation

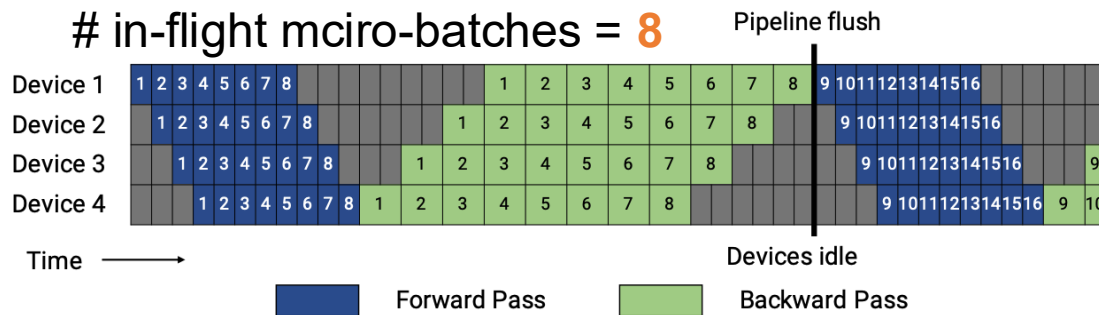


Can we improve the pipeline schedule to reduce memory requirement?

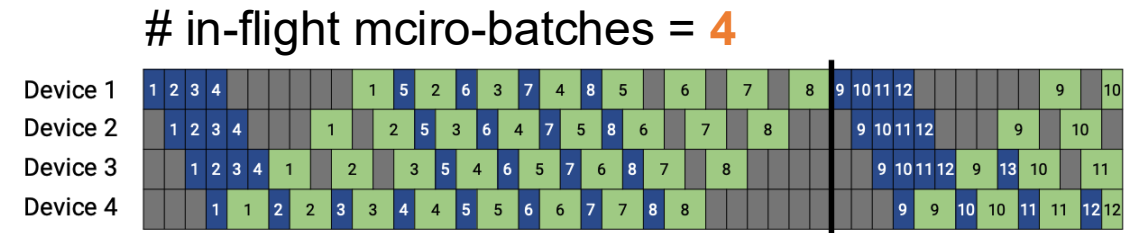
Pipeline Parallelism with 1F1B Schedule

- One-Forward-One-Backward in the steady state
- Limit the number of in-flight micro-batches to the pipeline depth
- Reduce memory footprint of pipeline parallelism
- Doesn't reduce pipeline bubble

Can we reduce pipeline bubble?



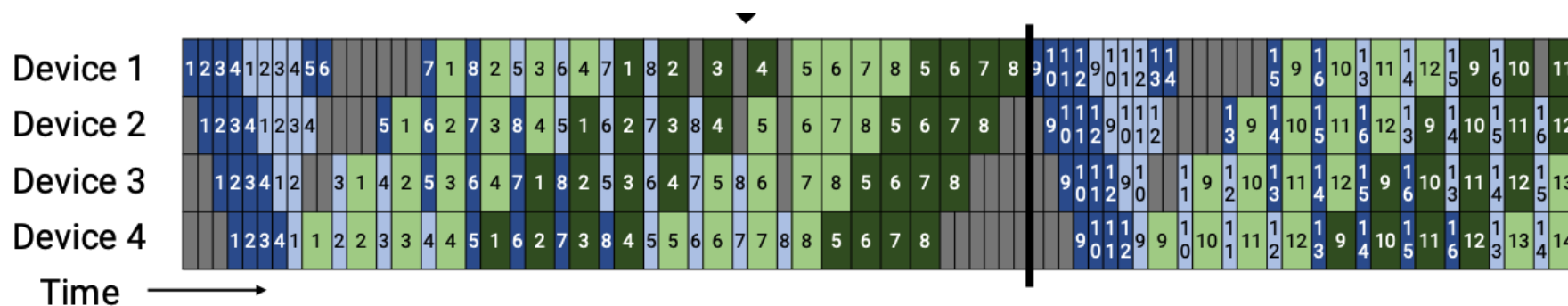
Pipeline parallelism with GPipe's schedule



Pipeline parallelism with 1F1B schedule

Pipeline Parallelism with Interleaved 1F1B Schedule

- Further divide each stage into ν sub-stages
- The forward (backward) time of each sub-stage is $\frac{t_f}{\nu}$ ($\frac{t_b}{\nu}$)



Each device is assigned two chunks. Dark colors show the first chunk and light colors show the second chunk.

$$\textit{BubbleFraction} = \frac{(p-1) * \frac{(t_f+t_b)}{v}}{m * t_f + m * t_b} = \frac{1}{v} * \frac{p-1}{m}$$

Reduce bubble time at the cost increased communication

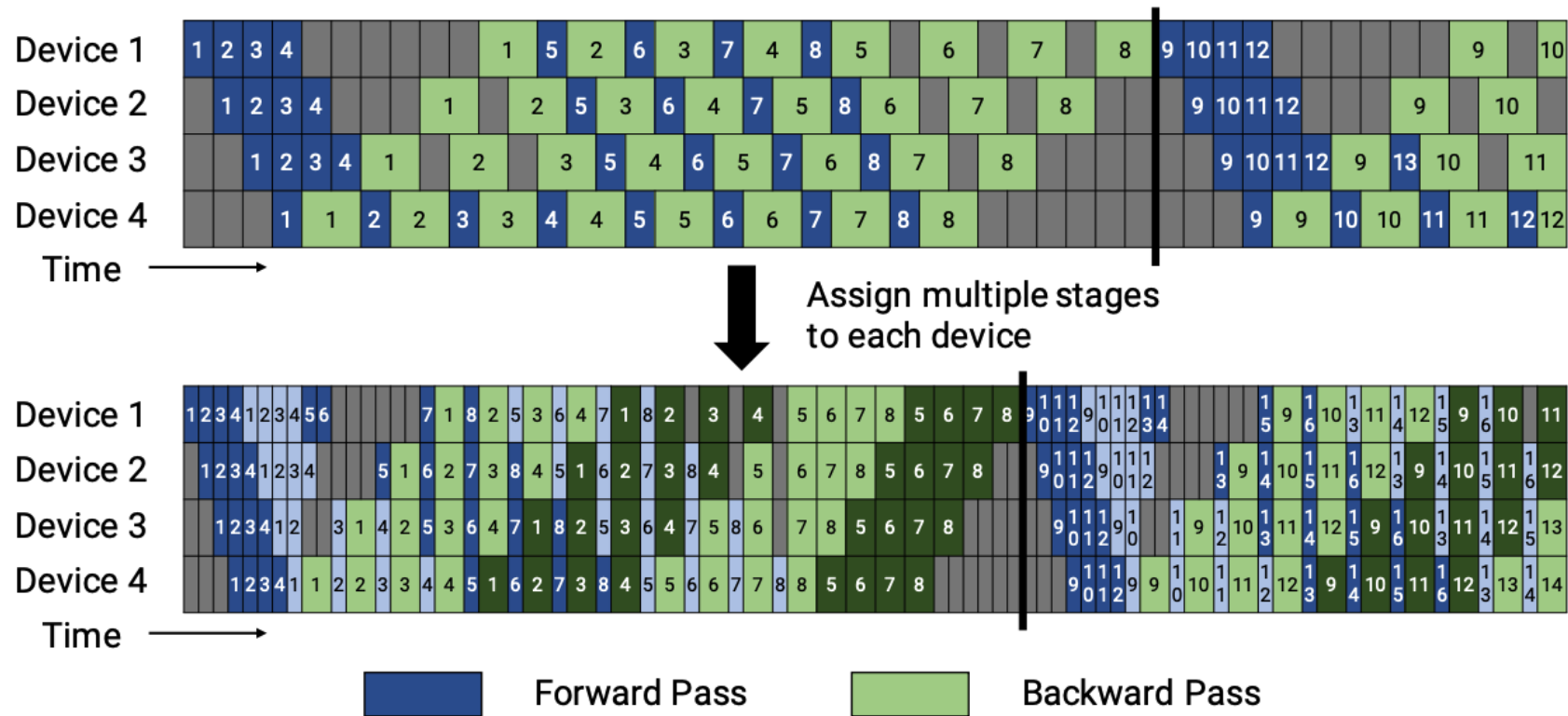
Pipeline Parallelism with Interleaved 1F1B Schedule

Pipeline parallelism with 1F1B Schedule

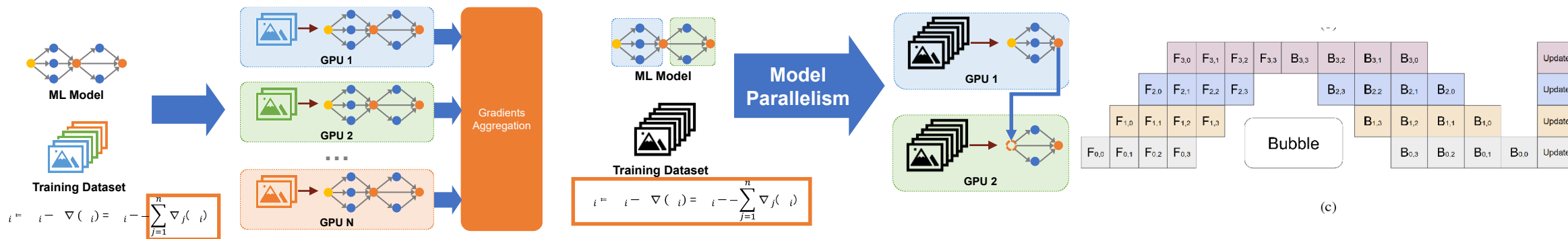
$$\text{BubbleFraction} = \frac{p-1}{m}$$

Pipeline parallelism with interleaved 1F1B Schedule

$$\text{BubbleFraction} = \frac{1}{v} * \frac{p-1}{m}$$

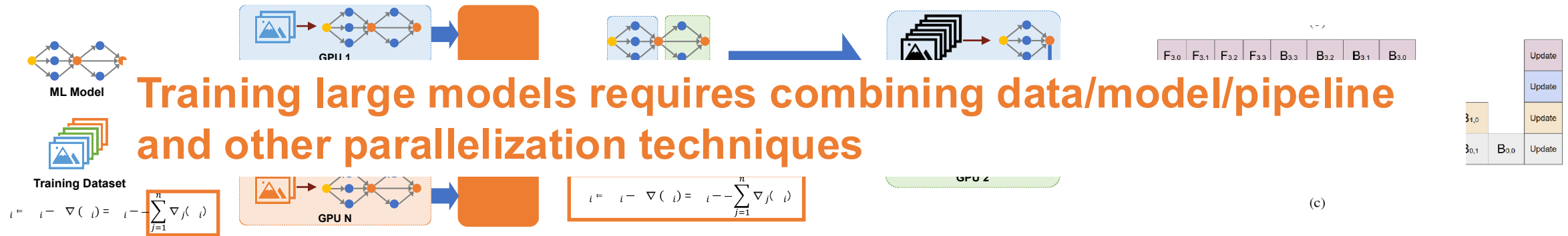


Summary: Comparing Data/Tensor Model/Pipeline Model Parallelism



	Data Parallelism	Tensor Model Parallelism	Pipeline Model Parallelism
Pros	✓ Massively parallelizable ✓ Require no communication during forward/backward	✓ Support training large models ✓ Efficient for models with large numbers of parameters	✓ Support large-batch training ✓ Efficient for deep models
Cons	❖ Do not work for models that cannot fit on a GPU ❖ Do not scale for models with large numbers of parameters	❖ Limited parallelizability; cannot scale to large numbers of GPUs ❖ Need to transfer intermediate results in forward/backward	❖ Limited utilization: bubbles in forward/backward

Summary: Comparing Data/Tensor Model/Pipeline Model Parallelism



	Data Parallelism	Model Parallelism	Pipeline Parallelism
Pros	✓ Massively parallelizable ✓ Require no communication during forward/backward	✓ Support training large models ✓ Efficient for models with large numbers of parameters	✓ Support large-batch training ✓ Efficient for deep models
Cons	❖ Do not work for models that cannot fit on a GPU ❖ Do not scale for models with large numbers of parameters	❖ Limited parallelizability; cannot scale to large numbers of GPUs ❖ Need to transfer intermediate results in forward/backward	❖ Limited utilization: bubbles in forward/backward

Example: 3D parallelism in DeepSpeed

Pipeline Model Parallelism

Data Parallelism

Tensor Model Parallelism

